

Harnesses and Multi-Agent Orchestration

From a LLM to a single agent to agentic teams

Sangwhan Moon

June 22, 2026

Who I Am

- Software Engineering Manager at Google, working on Chrome.
- Ph.D in artificial intelligence; thesis on Korean tokenization in LLMs.
- Part-time hobbyist researcher focused on LLM post-training, tokenization, and representation learning.

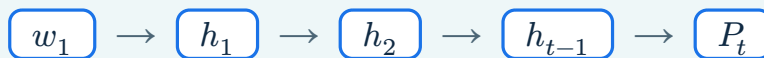
sangwhan.com/about

What Is a Language Model?

THE OBJECTIVE

- Estimate a distribution over the next word/token.
- Condition on the prefix observed so far.
- Generation samples, appends, and repeats.

RNN REGIME



prefix compressed into recurrent state

$$h_t = f(h_{t-1}, x_t)$$

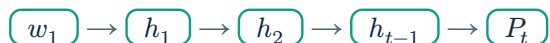
$$P_t = \text{softmax}(Wh_t)$$

$$P_{t(w_t \mid (w_1, \dots, w_{t-1}) \in \mathcal{V}^{t-1})}$$

From RNNs to Transformers

RNN BOTTLENECK

- Prefix flows through one recurrent state.
- Training is sequential across positions.
- Scaling is bottlenecked by recurrence and compression.



TRANSFORMER DELTA

- Tokens attend directly to other tokens.
- Positions train in parallel on accelerators.
- Scale becomes a practical lever.



Scale is the relevance: attention removed enough sequential bottleneck for predictable compute/data/model scaling to work.

Vaswani et al. 2017 [1]; Kaplan et al. 2020 [22]; Hoffmann et al. 2022 [23]

How Models Are Built

PRE-TRAINING

Broad next-token competence from large-scale data.

POST-TRAINING

SFT and preference optimization teach instruction-following behavior.

MID-TRAINING (OPTIONAL)

Common now: adapt behavior for code, math, long context, or domains.

PRODUCTION

Product systems wrap the model with policy, tools, and constraints.

Base Model to Agent

WHAT GETS ADDED

- Instruction tuning
- Tool schemas
- System prompt

WHY IT CHANGES BEHAVIOR

- Model asks; harness executes
- Harness runs the loop
- Side effects stay controlled

base model → SFT → tool use → system prompt → harness

What the Harness Owns

CONTROL LOOP

- Manages message history.
- Sends tool schemas to the model.
- Executes tool calls and appends results.

SECURITY BOUNDARY

- Ask, approve, block
- Model proposes actions
- Harness owns execution

Example basic harness: github.com/cynthia/kodex

Multi-Turn Tool Use

INTERACTION TRACE

```
user: "Find failing tests."  
assistant: tool_call(run_tests, {})  
tool: "2 failures in parser_test"  
assistant: tool_call(read_file, {path})  
tool: "parser edge case..."  
assistant: "Patch the parser..."
```

WHAT CHANGES

- Results become next-turn evidence
- Tool errors become state
- The harness appends ground truth
- Multi-turn drift can compound

ReAct (Yao et al., ICLR 2023) [5]; Toolformer (Schick et al., arXiv 2023) [20]

Single-Agent Harnesses Hit a Ceiling

THE BOTTLENECK

- One window
- One trajectory
- One lens
- Early mistakes compound

Your context window at 94%



LongBench (Bai et al., ACL 2024) [7]; LLMs Get Lost In Multi-Turn Conversation (Laban et al., ICLR 2026) [13]

Context Is Capacity, Not Control

A WINDOW HAS

- Instructions
- Files, logs, and tool output
- Prior mistakes and assumptions

A PLAN ADDS

- Ordering
- Dependencies
- State boundaries
- Explicit handoffs

Lost in the Middle (Liu et al., TACL 2024) [14]

Long Inputs Degrade Attention in Practice

NEEDLE TESTS

- Position-sensitive failures
- Longer windows still have cliffs

REAL TASKS

- LongBench spans QA, summary, code
- Compression helps, risk remains

CAVEAT

- Sparse attention changes scaling
- It does not create a plan
- Organization still matters

LongBench (Bai et al., ACL 2024) [7]; Lost in the Middle (Liu et al., TACL 2024) [14]

Long Agent Chains Accumulate Errors

CHAIN RISK

- New state every step
- Bad assumptions stick
- Early drift steers the run

MULTI-TURN RISK

- Reliability drops across turns
- Often interaction failure, not capability failure

TOOL CALL

reads files, runs command

→

WRITTEN STATE

summary + assumptions

→

NEXT TURN

state becomes context

step 1 → small drift → more drift → wrong branch

ReAct (Yao et al., ICLR 2023) [5]; LLMs Get Lost In Multi-Turn Conversation (Laban et al., ICLR 2026) [13]

Decomposition Is the Escape Hatch

WHAT WORKS

- Split the task
- Match role + context
- Pass forward only what matters

WHY MODELS ARE GOOD AT IT

- Strong at structure
- Good with intermediates
- L2M / ToT hint at the pattern

large task → **A** **B** **C** → **synthesis**

divide-and-conquer lowers interference; synthesis carries the risk

Least-to-Most (Zhou et al., ICLR 2023) [4]; Tree of Thoughts (Yao et al., NeurIPS 2023) [6]

Multi-Agents and Ensemble Inspiration

CORE CLAIM

- Test-time ensemble
- Roles + tools + memory boundaries
- Error diversity is the asset
- Diversity first, synthesis later

trajectory = $f(\text{model}, \text{role}, \text{context}, \text{tools})$

idealized: independent errors shrink as p^k

real agents are correlated; diversify context



Self-Consistency (Wang et al., ICLR 2023) [3]; More Agents Is All You Need (Li et al., TMLR 2024) [10]

Voting Beats One Guess When Errors Differ

SAMPLE DIVERSITY

- Self-Consistency samples multiple reasoning paths.
- It selects the answer that is most consistent across paths.
- Gains depend on task difficulty and error diversity.

AGENT FOREST

- More Agents Is All You Need scales the same intuition to instantiated agents.
- Parallel attempts are cheap when orchestration is automated.

path A path B path C → consistent answer

Self-Consistency (Wang et al., ICLR 2023) [3]; More Agents Is All You Need (Li et al., TMLR 2024) [10]

Specialization Can Beat One Generalist

ROLE FIT

- Different roles, different strengths
- Keep diversity to the merge
- Assign by failure mode

FUSION

- Rank candidates
- Fuse best fragments
- Late merge beats early collapse

architect qa security performance → synthesizer

LLM-Blender (Jiang et al., ACL 2023) [11]; Mixture-of-Agents (Wang et al., ICLR 2025) [12]

Debate Adds Friction

DELIBERATION

- Chain-of-thought exposes intermediate steps.
- Tree of Thoughts searches paths before committing.

GROUNDING

- ReAct interleaves reasoning and action.
- Collaboration externalizes deliberation across workers.

Chain-of-Thought (Wei et al., NeurIPS 2022) [2]; ReAct (Yao et al., ICLR 2023) [5]; Tree of Thoughts (Yao et al., NeurIPS 2023) [6]

Small Contexts Can Help

WHY IT WORKS

- If split preserves signal
- Smaller prompts, fewer distractions
- Narrower checklists
- Disagreements surface early

WHERE IT FAILS

- Weak synthesis = more text
- Orchestration decides reliability

SNR improves only when decomposition removes distractors without losing evidence

LongBench (Bai et al., ACL 2024) [7]; More Agents Is All You Need (Li et al., TMLR 2024) [10]; LLM-Blender (Jiang et al., ACL 2023) [11]; Mixture-of-Agents (Wang et al., ICLR 2025) [12]

Recent Industry Trends

SAKANA FUGU

Multi-agent system as one model API. Coordinates a model pool.

GEMINI DEEP RESEARCH

Plan → browse → refine → report. Agentic research with source links.

OPENROUTER FUSION

Parallel model panel plus judge. Consensus, gaps, and blind spots.

Industry signals, not proof of this architecture.

Sakana Fugu · Gemini Deep Research · OpenRouter Fusion Router

Where This Differs

MODEL COMPOSITION

- Merge or coordinate model capabilities
- Unit of work: model output
- Goal: stronger answer behind one API

RESEARCH AGENTS

- Plan, browse, summarize, cite
- Unit of work: research report
- Goal: better information synthesis

THIS HARNESS

- Run workers in disposable workspaces
- Unit of work: executable change
- Optimizes process reliability
- Goal: visible work, explicit synthesis

This narrows the scope from general agent composition to executable software work.

Design Around Failure

LOCKDOWN

Safe, but the human becomes the bottleneck.

YOLO

Fast, but one bad command can destroy the workspace.

REVERSIBLE

Agents get freedom inside a workspace that can roll back instantly.

Target: controlled blast radius, not constant approval.

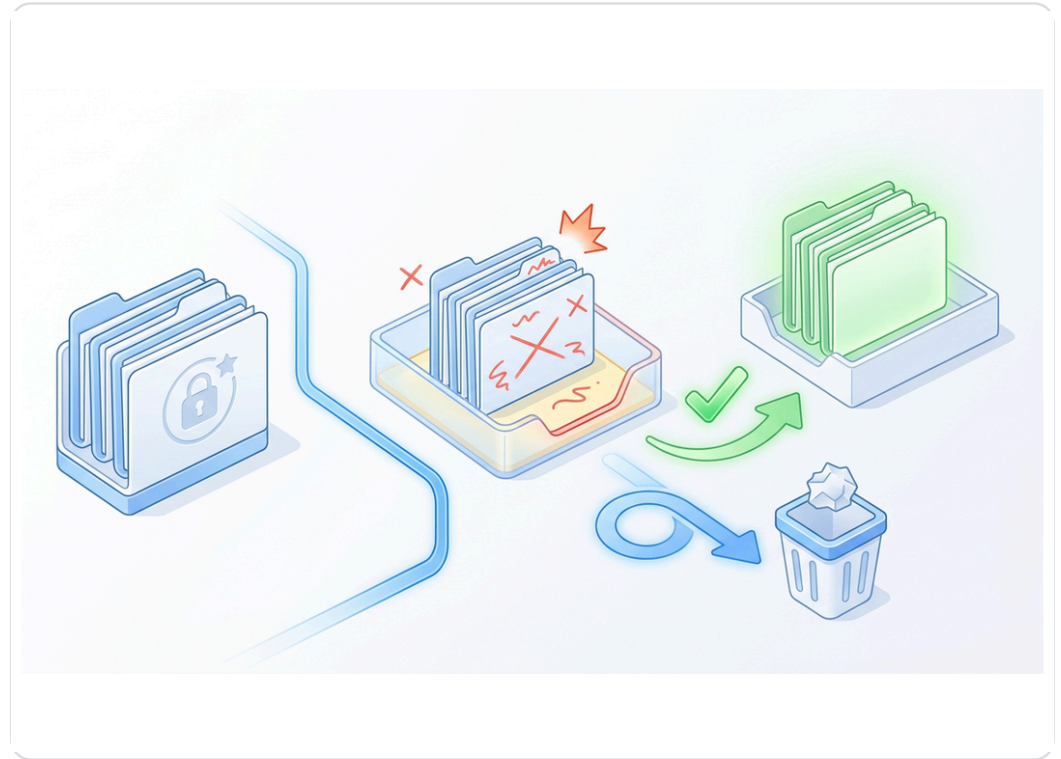
Safe-to-Destruct Workspaces

ZFS SNAPSHOTS

- Copy-on-write workspace
- Near-instant rollback

CHROOT

- Filesystem boundary
- Commit or rollback stays explicit



Agent Teams Are a DAG

PHASES

- Plan → phase → synthesize
- Dependencies stay explicit
- Handoffs are serialized state

PARALLELISM

- Concurrent tasks
- Bounded context
- Side effects stay bounded
- Reuse good work

`plan` → `phase 1 agents` → `synthesis` → `phase 2 agents` → `final`

Example

SOFTWARE TEAM

LEAD

plan + tradeoffs

BACKEND

API + data

FRONTEND

UI + state

QA

tests + security

Named workers get separate context, isolated execution, visible output, and auditable handoffs.

Inspired by Gastown's persistent identities, mailboxes, and handoffs.

1 REQUEST

feature / fix



2 PLANNER

phased JSON



3 REVIEW

human accepts plan

4 FAN OUT

parallel tasks



BACKEND

snapshot + tools

FRONTEND

snapshot + tools

QA

snapshot + tools

LEAD

constraints

5 MAILBOX

named handoffs



6 SYNTHESIZE

phase summary



7 CONTINUE

next phase or retry

final deliverable

rollback is available at each workspace boundary

Orchestrator + Mailbox Make a Team

LEADER / ORCHESTRATOR

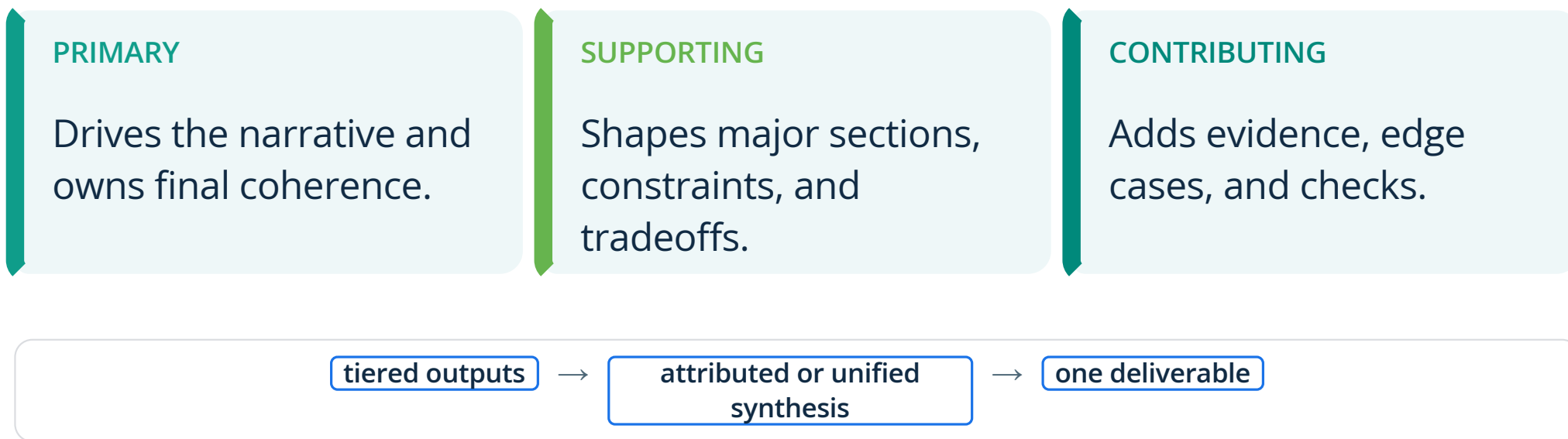
- Plans phases and roles
- Fans work out to agents
- Synthesizes and gates progress

INTER-AGENT MAIL

- Named handoffs between workers
- Visible in prompt context
- Auditable coordination, not hidden state

leader → phase agents → MAILBOX → synthesis + next phase

Synthesis Has Influence Tiers



LLM-Blender (Jiang et al., ACL 2023) [11]; Mixture-of-Agents (Wang et al., ICLR 2025) [12]

Iteration and Harness Abstraction

FOLLOW-UP LOOP

- Deliverable → replan → rerun
- Recursion mechanism for iteration
- Fresh agents, fresh context
- Retry only failed work

FOUR LAYERS

- TUI
- Orchestrator
- Agent manager
- Harnesses

Example Projects

TERMNES

`termnes`

CPU | PPU | mapper | frame

Output: NES emulator
CPU, PPU, mappers, renderer

Input: NES docs + one test ROM

github.com/cynthia/termnes

AT3RS

`at3rs`

bitstream → bands → PCM

Output: ATRAC3 codec
niche signal processing

Input: paper + ref codec via Wine + WAVs

github.com/cynthia/at3rs

SLOPPNG

`sloppng`

IHDR | IDAT | CRC | CLI

Output: PNG tooling
parser, validation, CLI

Input: W3C PNG specification

github.com/cynthia/sloppng

All three reached working implementations in one overnight run.

What the Results Demonstrate

DECOMPOSITION

Plan → phases →
executable work

SPECIALIZATION

Experts own formats,
rendering, QA

VERIFICATION

Review passes catch
integration bugs

PARALLEL

separate workers keep
context small

SYNTHESIS

best fragments become one
design

INTEGRATION

the output becomes a
coherent codebase

VERIFICATION

review catches cross-file
bugs

Why Not Open Source?

OPINIONATED

- One workflow
- Orthodox UI
- ZFS assumptions

BESPOKE ERA

- Harnesses age quickly
- Local loops beat frameworks
- Adapt before polish

TAKEAWAYS

- Design principles
- Fit your own workflow
- Build your own loop

The portable part is the mindset: cheap failure, visible work, explicit synthesis.

Questions?

Harnesses & Multi-Agent Orchestration

References

- [1] Ashish Vaswani et al., Attention Is All You Need. NeurIPS 2017.
- [2] Jason Wei et al., Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022.
- [3] Xuezhi Wang et al., Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023.
- [4] Denny Zhou et al., Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. ICLR 2023.
- [5] Shunyu Yao et al., ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.
- [6] Shunyu Yao et al., Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS 2023.
- [7] Yushi Bai et al., LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. ACL 2024.
- [8] Xiao Liu et al., AgentBench: Evaluating LLMs as Agents. ICLR 2024.
- [9] Shuyan Zhou et al., WebArena: A Realistic Web Environment for Building Autonomous Agents. ICLR 2024.
- [10] Junyou Li et al., More Agents Is All You Need. TMLR 2024.
- [11] Dongfu Jiang et al., LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion. ACL 2023.
- [12] Junlin Wang et al., Mixture-of-Agents Enhances Large Language Model Capabilities. ICLR 2025.
- [13] Philippe Laban et al., LLMs Get Lost In Multi-Turn Conversation. ICLR 2026.
- [14] Nelson F. Liu et al., Lost in the Middle: How Language Models Use Long Contexts. TACL 2024.
- [15] Sakana AI, Fugu. Product page.
- [16] Takuya Akiba et al., Evolutionary Optimization of Model Merging Recipes. arXiv 2024.
- [17] Chris Lu et al., The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. arXiv 2024.
- [18] Google, Gemini Deep Research. Product blog.
- [19] OpenRouter, Fusion Router. Documentation.
- [20] Timo Schick et al., Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv 2023.
- [21] Gastownhall, Gastown: multi-agent workspace manager. GitHub repository.
- [22] Jared Kaplan et al., Scaling Laws for Neural Language Models. arXiv 2020.
- [23] Jordan Hoffmann et al., Training Compute-Optimal Large Language Models. arXiv 2022.